Heurística construtiva e busca em vizinhança variável para a localização de locais de prova

Vinícius Latini Gonçalves[1], **Gustavo Bersan Moreira Campos**[1]

[1]Departamento de Ciência da Computação – Universidade Federal de Juiz de Fora (UFJF)
Juiz de Fora – MG – Brasil

vinicius.latini@estudante.ufjf.br (matrícula 202165149C)

bersan.gustavo@estudante.ufjf.br (matrícula 202435019)

Repositório: https://github.com/Bersan121/Alocacao_candidatos_PISM.git

1. Introdução

Todo ano, processos seletivos como o Programa de Ingresso Seletivo Misto (PISM) da UFJF precisam decidir em quais escolas aplicar as provas e para qual escola mandar cada candidato. Os candidatos fazem provas de tipos diferentes (módulos e áreas), uma sala não aplica dois tipos de prova ao mesmo tempo e cada escola tem um número limitado de lugares. Se a alocação for ruim, milhares de candidatos atravessam a cidade no dia da prova; uma alocação boa reduz deslocamentos, atrasos e custos de transporte.

[Oliveira et al. 2021] modelam o problema como uma variante do *Single Source Capacitated Facility Location Problem* (SSCFLP): cada candidato é atendido por um único local, os locais têm capacidade e cada local oferece um só tipo de prova. Aqui exigimos o tipo único de cada sala e deixamos cada escola oferecer até dois tipos, em salas diferentes (Seção 2.2). Para esse problema propomos uma heurística construtiva em duas fases, que sempre encontra uma solução viável quando a instância tem solução (Seção 5), e uma descida em vizinhança variável (VND) [Mladenović and Hansen 1997] com sete vizinhanças (Seção 6). Os resultados são comparados com um modelo de programação inteira e com um limite inferior (Seções 7 e 8). O código-fonte e os arquivos para reproduzir os experimentos estão em https://github.com/Bersan121/Alocacao_candidatos_PISM.git.

Ao longo do texto procuramos justificar cada decisão. A Seção 2 define o problema e a interpretação que adotamos, e a Seção 3 descreve as instâncias, cujas propriedades guiaram o projeto da heurística. Depois de apresentar o método e os resultados, mostramos um exemplo passo a passo (Seção 9), discutimos os casos em que a abordagem vai mal (Seção 10) e concluímos na Seção 11.

2. Descrição do problema

2.1. Elementos

Considere um conjunto de candidatos C ($|C| = M$), um conjunto de escolas L ($|L| = N$), cada escola j com um conjunto de salas R_j, sendo q_r a capacidade da sala r, e o conjunto de tipos de prova $T = \{\mathrm{M1}, \mathrm{M2}, \mathrm{M3e}, \mathrm{M3s}, \mathrm{M3h}, \mathrm{M3d}\}$. Cada candidato i tem um tipo de prova $\tau(i) \in T$, e d_{ij} é a distância do candidato i à escola j. A capacidade da escola é $Q_j = \sum_{r \in R_j} q_r$. Nenhuma escola vem com tipo definido; os tipos são decididos pelo modelo.

2.2. Interpretação adotada: salas com um tipo, escolas com até dois tipos

O enunciado diz que cada local de prova oferece apenas um tipo de exame. Adotamos duas regras. A primeira é que cada sala aplica um único tipo de prova, porque é assim que o processo funciona na prática: todos os candidatos de uma sala fazem a mesma prova, com o mesmo tempo e o mesmo fiscal. Nenhuma sala é dividida entre tipos.

A segunda é que cada escola oferece no máximo $K = 2$ tipos de prova, em salas diferentes, e os tipos da escola são os tipos das suas salas. A ideia é que cada escola tenha uma ou duas coordenações de prova. A logística continua simples, com no máximo dois conjuntos de provas e dois horários de término por escola, e a escola deixa de ficar presa a um único tipo, o que permite mandar mais candidatos para escolas perto de casa.

O limite K é um parâmetro do programa (`--tipos-por-escola`); todos os resultados deste relatório usam $K = 2$.

2.3. Formulação

Com as variáveis $x_{ij} = 1$ se o candidato i é alocado à escola j, $y_{jt} = 1$ se a escola j oferece o tipo t e $z_{rt} = 1$ se a sala r aplica a prova do tipo t, o problema é

$$\min \quad \sum_{i \in C} \sum_{j \in L} d_{ij}\, x_{ij} \tag{1}$$

$$\text{s.a.} \quad \sum_{j \in L} x_{ij} = 1 \qquad \forall i \in C \tag{2}$$

$$\sum_{t \in T} z_{rt} \le 1 \qquad \forall j \in L,\ r \in R_j \tag{3}$$

$$z_{rt} \le y_{jt} \qquad \forall j \in L,\ r \in R_j,\ t \in T \tag{4}$$

$$\sum_{t \in T} y_{jt} \le K \qquad \forall j \in L \tag{5}$$

$$\sum_{i \in C:\, \tau(i) = t} x_{ij} \le \sum_{r \in R_j} q_r\, z_{rt} \qquad \forall j \in L,\ t \in T \tag{6}$$

$$x_{ij},\ y_{jt},\ z_{rt} \in \{0, 1\}. \tag{7}$$

A restrição (2) manda cada candidato para exatamente uma escola, e (3) dá no máximo um tipo a cada sala. As restrições (4) e (5) limitam a K os tipos de cada escola. A restrição (6) cuida ao mesmo tempo da capacidade e da compatibilidade, pois candidatos do tipo t só ocupam salas do tipo t, até a soma das capacidades dessas salas. Basta escrevê-la por escola: salas do mesmo tipo numa mesma escola são intercambiáveis, e sempre existe um jeito de distribuir os candidatos entre elas respeitando a capacidade de cada uma. As duas decisões do enunciado aparecem como (i) quais escolas são usadas, e com quais tipos, e (ii) para onde vai cada candidato.

3. Instâncias

Usamos as instâncias do nosso grupo (pasta `instancias`, marcadas com ‡) e as instâncias oficiais da turma (pasta `instancias_pism`, um par por grupo, de G1 a G14). Todas seguem o mesmo formato de texto, e o leitor aceita também a variação do G6, que traz as duas contagens na mesma linha. Um arquivo do G7 continha apenas um caminho

de arquivo, sem dados, e foi descartado; arquivos com conteúdo repetido entram uma só vez, o que inclui as instâncias do nosso grupo, que são as do G3. Sobram 28 instâncias, listadas na Tabela 1, com 100 a 6798 candidatos e 5 a 20 escolas. No resto do texto nos referimos a elas pelos identificadores da tabela, numerados em ordem crescente de número de candidatos.

Tabela 1. Instâncias utilizadas (*Folga*: capacidade total acima do número de candidatos; *Viável*: existe solução com um tipo por sala e até dois tipos por escola).

ID	Arquivo	M	N	Salas	Folga (%)	Viável
I01	`Instancia1_G2`	100	5	20	5,0	não
I02	`Instancia2_G2`	109	5	12	11,0	sim
I03	`instancia1_G8`	316	12	24	17,7	sim
I04	`instancia2_G8`	316	12	24	17,7	sim
I05	`instancia_1_G13`	360	14	43	15,3	sim
I06	`instancia_2_G14`	362	12	39	5,8	sim
I07	`instancia_1_G14`	370	12	39	16,8	sim
I08	`instancia_2_G13`	382	16	44	7,6	sim
I09	`Instancia1_G7(1)`	3357	11	62	17,7	sim
I10	`Instancia1_G11`	3452	10	88	15,6	sim
I11	`Instancia1_G9`	3696	14	150	17,7	sim
I12	`instancia_2`‡	3858	10	97	19,5	sim
I13	`Instancia1_G12`	3860	15	205	16,2	sim
I14	`Instancia2_G12`	3860	15	205	16,2	sim
I15	`Instancia2_G4`	3970	12	142	17,6	sim
I16	`Instancia1_G4`	3995	12	145	17,6	sim
I17	`Instancia2_G9`	4124	14	150	5,5	sim
I18	`Instancia1_G5`	4213	15	117	10,7	sim
I19	`Instancia2_G1`	4442	15	157	18,4	sim
I20	`Instancia2_G11`	4454	15	122	23,4	sim
I21	`Instancia1_G1`	4518	15	125	6,9	sim
I22	`Instancia2_G5`	4628	15	125	8,1	sim
I23	`Instancia1_G10`	4744	15	101	10,1	sim
I24	`Instancia2_G6`	4809	15	177	7,5	sim
I25	`Instancia2_G10`	5116	15	101	2,1	sim
I26	`Instancia1_G6`	5203	15	248	11,1	sim
I27	`instancia_1`‡	5917	20	197	6,9	sim
I28	`Instancia2_G7`	6798	20	113	2,1	sim

Antes de projetar a heurística, analisamos as instâncias. Cinco propriedades orientaram as decisões:

1. A folga de capacidade é pequena, de 2,1% a 23,4%. Quase todas as escolas precisam ser usadas, então a decisão principal deixa de ser abrir ou fechar escolas e passa a ser quais tipos cada escola oferece.
2. Cobrir os seis tipos dá trabalho. As salas de cada tipo precisam somar capacidade para a demanda daquele tipo, e como as salas são indivisíveis isso é um problema de cobertura de *bins*, dificultado ainda pelo limite de dois tipos por escola. 27 das 28 instâncias têm solução; I01 não tem, embora a capacidade total sobre (Seção 10). Mesmo nas viáveis a cobertura costuma ficar justa, o que limita as mudanças de tipo.
3. A capacidade aperta. Se cada candidato fosse para a escola mais próxima, entre 39% e 100% deles cairiam numa escola lotada, com carga de até 12,1 vezes a capacidade. A alocação precisa levar a capacidade em conta desde o início, e as vizinhanças precisam trocar candidatos de lugar, já que quase não há vagas livres para onde movê-los.
4. Não há custo de abertura. O objetivo é só a distância, e usar uma sala a mais nunca

piora a solução, então podemos dar tipo a todas as salas, mesmo que algumas fiquem vazias.

5. As unidades das distâncias variam. Em algumas instâncias a matriz está em graus (valores perto de 0,1) e em outras em quilômetros, por isso comparamos os resultados pelo *gap* percentual em vez do valor absoluto.

4. Representação e função de avaliação

4.1. Representação

Uma solução é o par $(tipo, aloc)$, em que $tipo[j][r] \in \{-1\} \cup T$ é o tipo da sala r da escola j (-1 indica sala sem tipo) e $aloc[i] \in L$ é a escola do candidato i. Os tipos da escola j formam o conjunto $T_j = \{tipo[j][r] \geq 0\}$, e a solução precisa ter $|T_j| \leq K$. Junto com o par, mantemos de forma incremental o custo total e duas matrizes $N \times |T|$: $cap[j,t]$, a soma das capacidades das salas de tipo t na escola j, e $carga[j,t]$, o número de candidatos do tipo t alocados à escola j. A solução é viável se e somente se $|T_j| \leq K$ e $carga[j,t] \leq cap[j,t]$ para todo (j,t).

Guardamos o tipo de cada sala, e não só os tipos da escola, porque é a divisão das salas entre os dois tipos que define a capacidade de cada um. Já a sala de cada candidato fica de fora da representação. Ela é definida no final, enchendo as salas de cada par (escola, tipo) da maior para a menor, o que usa o menor número de salas. Com isso as duas decisões do problema (tipos e alocação) ficam separadas, e mover um candidato custa $O(1)$, pois só muda $aloc[i]$, duas posições de $carga$ e o custo. Guardar a sala do candidato não mudaria o custo de nenhuma solução e só multiplicaria o número de vizinhos equivalentes.

4.2. Função de avaliação

$$f(tipo, aloc) = \sum_{i \in C} d_{i,aloc[i]}. \tag{8}$$

Não usamos penalização. O construtivo só gera soluções viáveis (Seção 5), e a busca local só aceita movimentos que mantêm a viabilidade (Seção 6), de modo que nenhuma restrição é violada em momento algum e não há parâmetro de penalidade para calibrar. Pudemos fazer assim porque checar a viabilidade de um movimento é barato neste problema: um candidato que muda de escola altera $carga$ em apenas duas posições, e uma mudança de tipo mexe nos tipos de uma ou duas escolas, cujo limite $|T_j| \leq K$ é conferido antes de avaliar o vizinho.

Se precisássemos aceitar soluções inviáveis (para trocar o tipo de uma escola deixando, por um tempo, candidatos sem lugar, por exemplo), usaríamos $f' = f + \rho \sum_{j,t} \max(0, carga[j,t] - cap[j,t])$ com $\rho > \max_{i,j} d_{ij}$. Com esse valor de ρ, levar um candidato excedente para qualquer vaga sempre compensa, e o ótimo de f' coincide com o ótimo viável.

Além disso, toda solução produzida nos experimentos passa por uma verificação independente, que recalcula do zero T_j, cap, $carga$, a distribuição nas salas e o custo. As soluções iniciais e finais das 270 execuções passaram nessa verificação.

5. Heurística construtiva

O construtivo tem duas fases. A fase A decide os tipos de cada escola e de cada sala e garante que cada tipo tenha capacidade suficiente; a fase B aloca os candidatos. A fase A é determinística, e a fase B usa uma lista restrita de candidatos (RCL) de tamanho k, como na fase de construção do GRASP [Feo and Resende 1995].

5.1. Fase A: tipos das escolas e das salas

O Algoritmo 1 resume a fase A.

Algoritmo 1 Fase A: tipos das escolas e das salas.

1 $d[j,t] \leftarrow 0$; ordena os candidatos pelo arrependimento entre as duas escolas mais próximas
2 **para cada** candidato i nessa ordem: coloca i na escola mais próxima com vaga (capacidade Q_j), $d[j,\tau(i)] \leftarrow d[j,\tau(i)] + 1$
3 **para cada** escola j: $T_j \leftarrow$ os dois tipos de maior $d[j,t]$ (só os com $d[j,t] > 0$)
4 **para cada** escola j:
5 **enquanto** houver sala livre e $\max_{t \in T_j}(d[j,t] - cap[j,t]) > 0$:
6 $t \leftarrow$ esse tipo; $\phi \leftarrow$ essa falta; $r \leftarrow$ menor sala livre com $q_r \geq \phi$ (se não houver, a maior livre); $tipo[j][r] \leftarrow t$
7 **para cada** sala ainda sem tipo (maiores primeiro): recebe, entre os tipos de T_j (ou entre todos, se $|T_j| < 2$), o de maior déficit global $\delta_t = dem_t - \sum_j cap[j,t]$ ou, sem déficit, o de maior falta local
8 **enquanto** existir t com $\delta_t > 0$:
9 candidatos: uma sala de tipo $u \neq t$ (se a escola continua com até dois tipos) ou todas as salas de u numa escola, desde que $-\delta_u$ seja maior ou igual às vagas mudadas
10 **se** houver: muda para t a da escola com maior falta local de t (empate: menos vagas mudadas)
11 **senão**: reparo exato (texto); se ele não tem solução, a instância é inviável

Para escolher os tipos de cada local, a fase A primeiro faz uma alocação provisória que ignora os tipos e respeita apenas a capacidade da escola (linhas 1–2). Ela serve para estimar quantos candidatos de cada tipo "querem" cada escola. Cada escola fica com os dois tipos de maior demanda estimada (linha 3), e suas salas são distribuídas entre eles (linhas 4–6) pela regra *best-fit*: a menor sala que cobre a falta do tipo. O critério de seleção é, portanto, a proximidade entre a escola e os candidatos de cada tipo, medida pela alocação provisória, e as salas acabam divididas entre os dois tipos mais ou menos na proporção da demanda estimada. As salas que sobram também recebem um tipo (linha 7), já que abrir uma sala nunca piora a solução e dá mais espaço para a fase B e para a busca local.

Ao fim da fase A, a capacidade de cada tipo precisa ser pelo menos igual à sua demanda; se não for, a fase B trava com candidatos sem sala. As linhas 8–11 tratam dessa cobertura. Uma sala, ou um tipo inteiro de uma escola, só passa de u para t se o tipo u continua coberto sem ela, e assim cada mudança reduz o déficit total e o laço termina. Mudar todas as salas de u de uma vez resolve o caso em que a escola já tem dois tipos, pois aí uma sala sozinha não pode ir para t sem criar um terceiro tipo na escola.

5.2. Fase B: alocação dos candidatos

Os candidatos são considerados em ordem decrescente de arrependimento: quem mais perde se não ficar na escola mais próxima escolhe primeiro, numa ideia parecida com

Algoritmo 2 Fase B: alocação por arrependimento estático com RCL.

1 **para cada** candidato i: $pref_i \leftarrow$ escolas ordenadas por d_{ij} crescente
2 **para cada** candidato i: $arr_i \leftarrow d_{i,pref_i[2]} - d_{i,pref_i[1]}$
3 *fila* $\leftarrow$ candidatos em ordem decrescente de arr (calculada uma única vez)
4 $RCL \leftarrow$ os k primeiros da *fila*
5 **enquanto** $RCL \neq \emptyset$:
6 sorteia $i \in RCL$ com probabilidade uniforme; retira i da RCL
7 $j \leftarrow$ primeira escola de $pref_i$ com sala do tipo $\tau(i)$ e vaga
8 $r \leftarrow$ maior sala de j do tipo $\tau(i)$ com vaga; $aloc[i] \leftarrow j$; desconta uma vaga de r
9 insere na RCL o próximo candidato ainda não considerado da *fila*

a do método de aproximação de Vogel para o problema de transporte. Quem tem duas escolas próximas perde pouco se for deslocado e fica para o fim. O arrependimento é calculado uma única vez, o que mantém a fase em $O(MN \log N)$.

Cada candidato vai para a escola mais próxima que ainda tem vaga numa sala do seu tipo e ocupa a maior sala desse tipo com vaga. A capacidade é controlada sala a sala: a vaga é descontada da sala, e uma escola só é aceita se as suas salas do tipo do candidato ainda têm vaga somada positiva. A compatibilidade vem do fato de que só entram na busca salas do tipo do candidato, então ninguém é colocado numa sala de outro tipo.

O sorteio dentro da RCL dá diversidade às execuções. Com $k = 1$ o construtivo é guloso e determinístico; usamos $k = 10$, e a Seção 8.5 testa outros valores.

5.3. Escolhas atrativas que travam a construção

Uma construção puramente gulosa trava com facilidade neste problema. Se cada escola recebe os tipos de maior demanda local, um tipo com poucos candidatos espalhados (como M3s nas instâncias pequenas) pode não ser escolhido por escola nenhuma, ou ficar com menos vagas que candidatos, e os últimos candidatos desse tipo ficariam sem lugar. Por isso a fase A resolve a cobertura antes de alocar: quando ela termina, $\sum_j cap[j,t] \geq dem_t$ para todo t. Como $pref_i$ contém todas as escolas, a fase B sempre acha uma sala do tipo do candidato com vaga, seja qual for a ordem dos sorteios.

A própria cobertura gulosa (linhas 8–10) também pode travar. Com salas indivisíveis e no máximo dois tipos por escola, pode acontecer de nenhuma sala, e nenhum tipo inteiro de uma escola, poder mudar sem descobrir o tipo que atendia, mesmo que outra distribuição dos tipos entre as salas cubra todos. Para esse caso a fase A tem um reparo exato (linha 11). É um modelo inteiro pequeno, só com salas e tipos (sem candidatos e sem distâncias), que muda o tipo do menor total de vagas possível em relação à atribuição atual, respeitando as restrições (3)–(5) e a cobertura $\sum_j cap[j,t] \geq dem_t$. O modelo tem algumas centenas de variáveis binárias, e o HiGHS [Huangfu and Hall 2018] o resolve em frações de segundo. Escolhemos esse caminho porque o reparo só resolve a viabilidade: ele preserva o que a fase A decidiu e muda apenas o necessário, e quem continua decidindo com base na distância é a heurística. Quando o modelo não tem solução, temos uma prova de que a instância é inviável, e o método avisa sem gastar tempo de busca. A cobertura gulosa mudou o tipo de alguma sala em 24 das 27 instâncias viáveis, e o reparo exato foi necessário em 3 delas (I02, I25, I28).

6. Estruturas de vizinhança e busca local

6.1. Vizinhanças

Escolhemos as vizinhanças a partir da representação e das propriedades da Seção 3. Com folga pequena, quase toda escola está cheia e raramente dá para mover um candidato sozinho, então precisamos trocar candidatos ou encadear movimentos. E, como a representação separa os tipos da alocação, há vizinhanças para cada uma dessas decisões.

As três primeiras vizinhanças mexem só na alocação e mantêm os tipos:

N1 (shift). Move um candidato para outra escola que tenha vaga numa sala do seu tipo. Cada vizinho é avaliado em $O(1)$, e uma passada completa custa $O(MN)$. É a vizinhança mais simples e aproveita as vagas que as outras vizinhanças deixam livres.

N2 (swap). Troca dois candidatos do mesmo tipo entre as escolas j e k. Como a carga não muda, a troca é sempre viável, o que a torna a vizinhança natural para escolas cheias. A melhor troca de um par (j, k) sai em $O(|I_j| + |I_k|)$, como $\min_{a \in I_j}(d_{ak} - d_{aj}) + \min_{b \in I_k}(d_{bj} - d_{bk})$.

N3 (cadeias de ejeção). Generaliza N1 e N2 [Glover 1996]. O candidato i_1 vai de j_1 para j_2, i_2 de j_2 para j_3, e assim por diante, até voltar a j_1 ou chegar a uma escola com vaga. Para cada tipo t montamos um grafo cujos nós são as escolas com sala do tipo t e um nó auxiliar S, em que o arco $j \to k$ custa $\min_{i \in (j,t)}(d_{ik} - d_{ij})$. Um ciclo de custo negativo, encontrado por Bellman-Ford em $O(N^3)$, corresponde a uma cadeia de realocações que melhora f. Se não há ciclo negativo, a alocação é ótima para os tipos atuais, pela condição de otimalidade do problema de transporte [Ahuja et al. 1993].

As outras quatro mudam tipos, e só geram vizinhos com $|T_j| \leq 2$:

N4 (retipar sala). Muda o tipo de uma sala de u para t. O movimento só é permitido se a escola continua com até dois tipos, isto é, se t já está na escola, se a sala é a única de u ou se a escola tem um só tipo. Numa escola que já tem dois tipos, ele passa uma sala de um tipo para o outro.

N5 (trocar salas). Troca os tipos de duas salas da mesma escola, de tipos $u \neq t$ e capacidades $q_1 > q_2$. Com isso $q_1 - q_2$ vagas passam de u para t naquela escola, sem mudar T_j. É um ajuste fino da divisão entre os dois tipos da escola, útil quando tirar uma sala inteira de u com N4 deixaria esse tipo sem vagas suficientes.

N6 (tipo da escola). Todas as salas do tipo u na escola j passam para o tipo t, ou seja, a escola troca um dos seus tipos.

N7 (trocar escolas). As escolas j e k trocam um tipo: as salas de u em j passam para t, e as de t em k passam para u. Como a capacidade total de cada tipo quase não muda, dá para reorganizar os tipos mesmo quando nenhum tipo tem sobra para perder um conjunto de salas, o que é comum com a cobertura justa (Seção 3, propriedade 2). Há $O(N^2)$ vizinhos por passada, já que cada escola tem no máximo dois tipos.

Todas as vizinhanças de tipos alteram cap em algumas células (escola, tipo), e todas são avaliadas do mesmo jeito. Primeiro vêm as chegadas: nas células que ganham vagas, entram os candidatos daquele tipo, alocados em outras escolas, que mais ganham com a mudança ($d_{i,aloc[i]} - d_{ij} > 0$), até ocupar as vagas livres. Depois vêm os despejos: nas células que perdem vagas, o excedente sai, começando por quem perde menos, e

cada um vai para a escola mais próxima com vaga do seu tipo. O vizinho é aceito se $\Delta =$ custo dos despejos $-$ ganho das chegadas < 0 e descartado se faltar vaga para algum despejo, o que mantém a viabilidade. Salas iguais (mesma capacidade e mesmo tipo) geram o mesmo vizinho e são avaliadas uma vez só.

6.2. Estratégia de busca: VND

As vizinhanças são combinadas numa descida em vizinhança variável (VND) [Mladenović and Hansen 1997], na ordem N1 a N7 (Algoritmo 3), das mais baratas e locais para as mais caras e que mudam mais a estrutura da solução. N1, N2 e N3 são aplicadas até chegar ao ótimo local da vizinhança. Em N4 a N7 aplicamos o primeiro movimento que melhora (*first improvement*), porque cada mudança de tipo altera as capacidades e pede uma nova otimização da alocação, que é feita ao voltar para N1.

Algoritmo 3 VND.

$\mathcal{N} \leftarrow (N1, N2, \ldots, N7)$; $\ell \leftarrow 1$
enquanto $\ell \leq |\mathcal{N}|$:
  $s' \leftarrow$ busca local em $\mathcal{N}_\ell$ a partir de s
  se $f(s') < f(s)$: $s \leftarrow s'$; $\ell \leftarrow 1$ **senão**: $\ell \leftarrow \ell + 1$
devolve s (ótimo local em relação a todas as vizinhanças)

7. Metodologia experimental

Os experimentos rodaram em Python 3.13.5 com NumPy 2.5.4, num Apple M1 Max com 32 GB de memória (macOS 26.6), em uma única *thread*. O código e as instruções para reproduzi-los estão no repositório `https://github.com/Bersan121/Alocacao_candidatos_PISM.git` (`src/experimentos.py -t 2`).

Cada instância foi resolvida 10 vezes com $k = 10$, com as sementes 0 a 9. Em cada execução registramos o valor da solução inicial e o valor após o VND, a melhoria absoluta e percentual, o tempo do construtivo e da busca, o número de escolas e de salas usadas, e o número de movimentos e o ganho de cada vizinhança. Como o método tem componentes aleatórios, relatamos para cada instância a média, o desvio padrão, o melhor e o pior valor das 10 execuções, como pede o enunciado. Ao todo são 27 instâncias viáveis e 270 execuções.

Para medir a qualidade, resolvemos o modelo (1)–(6) com o HiGHS [Huangfu and Hall 2018], via SciPy [Virtanen et al. 2020], com limite de 60 s por instância e com as salas iguais de cada escola agregadas para reduzir a simetria. O MIP prova o ótimo em 7 das 27 instâncias e, em 1 instância grande, não encontra nenhuma solução em 60 s. Nas instâncias sem ótimo provado usamos duas referências. Uma é o melhor valor conhecido, o menor entre o MIP, a nossa heurística e uma alocação ótima com os tipos do construtivo. A outra é um limite inferior (LB), o maior entre o limite do MIP e uma relaxação em que a capacidade da escola é compartilhada livremente entre os tipos, que se resolve como um problema de transporte. O *gap* de um valor v é $100\,(v - v^\star)/v^\star$, sendo $v^\star$ a referência. Quando a referência é o melhor valor conhecido e foi a própria heurística que o encontrou, o gap dá zero mesmo sem prova de otimalidade. Por isso reportamos também o gap até o LB, que é uma garantia.

8. Resultados

8.1. Solução inicial e após a busca local

As Tabelas 2 e 3 trazem os resultados por instância, e a Figura 1 resume a qualidade. O construtivo fica, em média, 19,96% acima do melhor valor conhecido. O VND melhora a solução inicial em 15,29%, em média, e termina 0,73% acima do melhor valor conhecido. Em 24 das 27 instâncias o resultado final fica a menos de 1% da referência, e só 1 fica 5% ou mais acima.

Esse gap médio precisa ser lido com cuidado. A heurística encontra soluções melhores que o MIP de 60 s em 17 instâncias e, em 18 delas, ela mesma é a referência, sem prova de otimalidade, o que zera o gap. Nas 7 instâncias com ótimo provado, o VND chega ao ótimo em 1 e fica, no pior caso, 8,3% acima dele (I07; ver Seção 10).

O gap até o LB é uma garantia: 9,40% em média, com mediana de 4,42%. A média sobe por causa de I25 e I26 (105,2 e 47,8%). Nelas o MIP não melhora o LB de transporte, que ignora os tipos e fica fraco. Sem essas duas instâncias, a média cai para 4,04%.

Tabela 2. Valor da solução inicial (construtivo) e após o VND nas 10 execuções (média, desvio padrão, melhor e pior).

	Construtivo (10 execuções)				Após o VND (10 execuções)			
ID	média	desvio	melhor	pior	média	desvio	melhor	pior
I02	951,64	0,23	951,35	952,16	913,13	0,00	913,13	913,13
I03	1064,93	2,61	1059,85	1068,24	865,52	0,00	865,52	865,52
I04	107,95	0,02	107,90	107,99	107,48	0,00	107,48	107,48
I05	1291,20	3,58	1285,25	1297,33	1002,08	0,00	1002,08	1002,08
I06	2014,25	0,96	2012,62	2015,98	1868,06	0,00	1868,06	1868,06
I07	1454,35	1,50	1451,83	1456,23	1182,63	0,00	1182,63	1182,63
I08	1207,31	3,99	1199,90	1215,37	995,56	0,00	995,56	995,56
I09	19400,52	12,68	19383,59	19429,78	16495,78	0,00	16495,78	16495,78
I10	35443,23	9,20	35429,92	35457,10	25340,44	0,00	25340,44	25340,44
I11	7573,57	2,02	7570,62	7576,39	5870,99	0,00	5870,99	5870,99
I12	705,46	0,01	705,45	705,47	649,58	0,00	649,58	649,58
I13	17302,05	3,35	17297,80	17306,62	15382,63	0,00	15382,63	15382,63
I14	95301,63	0,22	95301,31	95302,02	92154,42	0,00	92154,42	92154,42
I15	9936,27	0,85	9935,34	9937,92	8100,75	0,00	8100,75	8100,75
I16	11175,08	4,01	11170,59	11182,37	8429,94	0,00	8429,94	8429,94
I17	12861,96	4,02	12854,72	12866,37	11823,63	0,00	11823,63	11823,63
I18	13482,64	3,95	13476,95	13488,84	10502,73	0,00	10502,73	10502,73
I19	13659,77	4,71	13654,75	13669,97	11161,55	0,00	11161,55	11161,55
I20	104691,50	5,34	104683,80	104701,13	102190,57	0,00	102190,57	102190,57
I21	29600,57	5,09	29591,82	29606,58	24150,26	0,00	24150,26	24150,26
I22	15787,23	6,48	15778,31	15797,95	12842,92	0,00	12842,92	12842,92
I23	9907,73	3,23	9902,66	9913,59	7369,43	0,00	7369,43	7369,43
I24	26563,28	9,14	26547,46	26579,34	22705,46	0,00	22705,46	22705,46
I25	13548,80	2,14	13544,94	13552,36	9923,36	0,00	9923,36	9923,36
I26	13479,53	2,00	13474,63	13481,40	10815,79	0,00	10815,79	10815,79
I27	1030,76	0,03	1030,73	1030,82	979,92	0,21	979,32	979,99
I28	70848,67	8,85	70837,39	70869,13	61663,84	0,00	61663,84	61663,84

8.2. Contribuição de cada vizinhança

A Tabela 4 e a Figura 2 mostram quanto cada vizinhança contribui. A N2 (swap) responde pela maior parte da melhoria (57,2%), como esperávamos pela folga pequena: com as escolas cheias, o ganho vem de trocar candidatos. A N1 quase não atua logo depois do construtivo, porque a fase B já põe cada candidato na escola mais próxima com vaga, mas volta a ser útil depois das mudanças de tipo, que abrem vagas novas. A N3 contribui

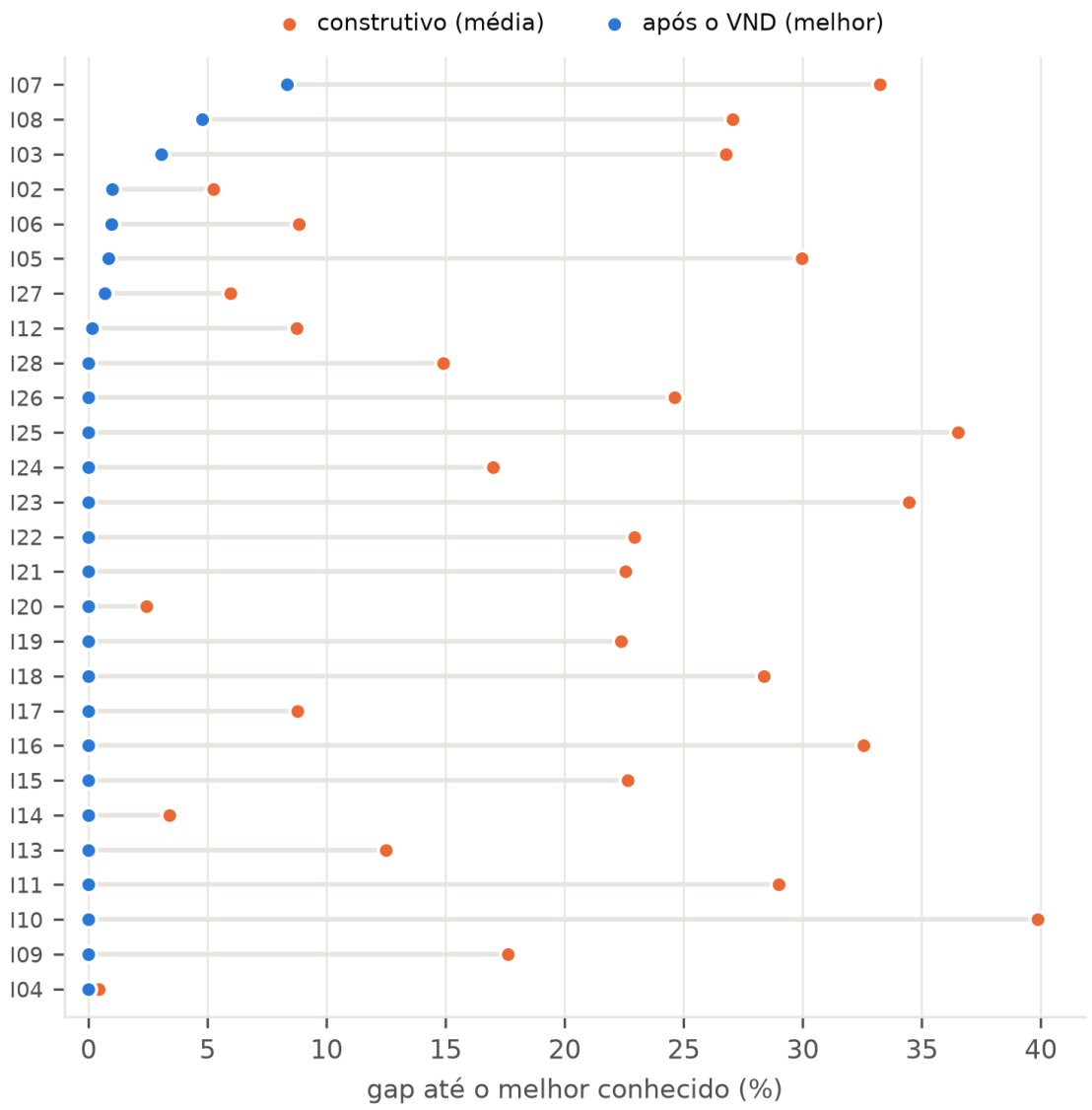


Figura 1. Gap até o melhor valor conhecido da solução inicial (média de 10 execuções) e da melhor solução após o VND, por instância.

Tabela 3. Melhoria média da busca local, gaps da solução inicial (média) e final (melhor execução) até a referência, gap final até o limite inferior, tempo médio por execução (t) e escolas com dois tipos e usadas na melhor solução. *: referência com otimalidade provada.

ID	Melhoria (%)	Gap ini. (%)	Gap final (%)	Gap LB (%)	t (s)	Escolas com 2 tipos	Escolas usadas
I02	4,05	5,25	0,99*	0,99	0,05	5/5	5/5
I03	18,72	26,78	3,04*	3,04	0,17	11/12	12/12
I04	0,43	0,43	0,00*	0,00	0,01	6/12	12/12
I05	22,39	29,95	0,85	2,09	0,20	14/14	14/14
I06	7,26	8,85	0,95*	0,95	0,06	7/12	12/12
I07	18,68	33,24	8,35*	8,35	0,10	12/12	12/12
I08	17,54	27,06	4,78*	4,78	0,12	15/16	16/16
I09	14,97	17,61	0,00	10,26	0,49	11/11	11/11
I10	28,50	39,87	0,00	4,42	5,93	10/10	10/10
I11	22,48	29,00	0,00	1,90	2,58	14/14	14/14
I12	7,92	8,75	0,14*	0,14	1,85	10/10	10/10
I13	11,09	12,48	0,00	1,89	1,19	15/15	15/15
I14	3,30	3,42	0,00	1,00	1,26	15/15	15/15
I15	18,47	22,66	0,00	5,47	1,63	12/12	12/12
I16	24,56	32,56	0,00	9,91	1,88	12/12	12/12
I17	8,07	8,78	0,00	3,58	0,93	14/14	14/14
I18	22,10	28,37	0,00	5,62	6,44	15/15	15/15
I19	18,29	22,38	0,00	5,12	2,93	15/15	15/15
I20	2,39	2,45	0,00	0,04	5,86	14/15	14/15
I21	18,41	22,57	0,00	6,42	1,09	15/15	15/15
I22	18,65	22,93	0,00	5,20	0,85	15/15	15/15
I23	25,62	34,44	0,00	6,94	4,57	15/15	15/15
I24	14,52	16,99	0,00	4,17	6,35	15/15	15/15
I25	26,76	36,53	0,00	105,15	2,32	14/15	15/15
I26	19,76	24,63	0,00	47,80	6,62	15/15	15/15
I27	4,93	5,96	0,67	0,68	6,48	20/20	20/20
I28	12,96	14,90	0,00	7,93	7,05	20/20	20/20

pouco em valor (3,1%), mas é ela que garante que a alocação final é ótima para os tipos encontrados.

Entre as vizinhanças de tipos, a N4 é a mais importante e a segunda no geral (18,0%). Passar uma sala de um tipo para o outro dentro da escola corrige a divisão de vagas que a fase A fez a partir de uma demanda só estimada. Depois vem a N7 (10,0%), que troca tipos entre escolas sem desfazer a cobertura. N5 e N6 contribuem pouco (2,6% e 4,4%), mas fazem movimentos que as outras não fazem: a N5 move menos vagas que uma sala inteira, e a N6 troca um tipo inteiro da escola quando ele tem sobra.

Tabela 4. Movimentos aplicados por execução, ganho por execução (em % do valor inicial) e participação de cada vizinhança na melhoria total (médias por instância).

Vizinhança	Movimentos	Ganho	% da melhoria
N1 shift	177,7	0,83%	4,8
N2 swap	1627,9	8,62%	57,2
N3 cadeias	204,2	0,49%	3,1
N4 retipar sala	19,8	3,02%	18,0
N5 trocar salas	21,8	0,40%	2,6
N6 tipo da escola	1,9	0,63%	4,4
N7 trocar escolas	3,6	1,30%	10,0

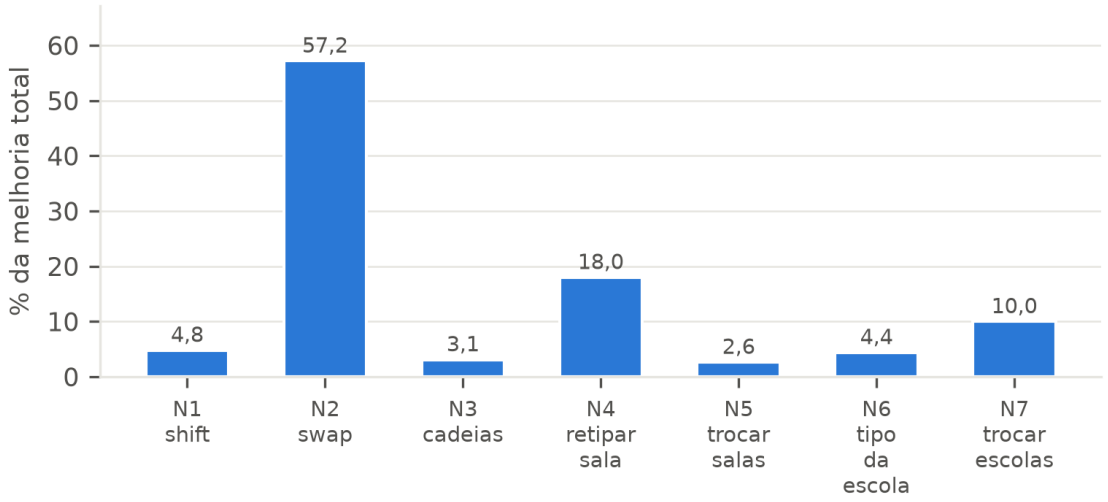


Figura 2. Participação de cada vizinhança na melhoria obtida pelo VND.

8.3. Tempo de execução

O construtivo leva no máximo 2,957 s, já contando o reparo exato quando ele é usado. O tempo total (Figura 3) vem quase todo do VND: 2,56 s em média e 7,05 s na instância mais lenta (I28). O tempo da busca cresce com o número de candidatos (correlação de 0,65 com M, por execução), mas acompanha mais de perto o número de mudanças de tipo aceitas em N4 a N7 (correlação de 0,90). Cada mudança aceita reinicia a VND em N1, que volta a percorrer todos os candidatos.

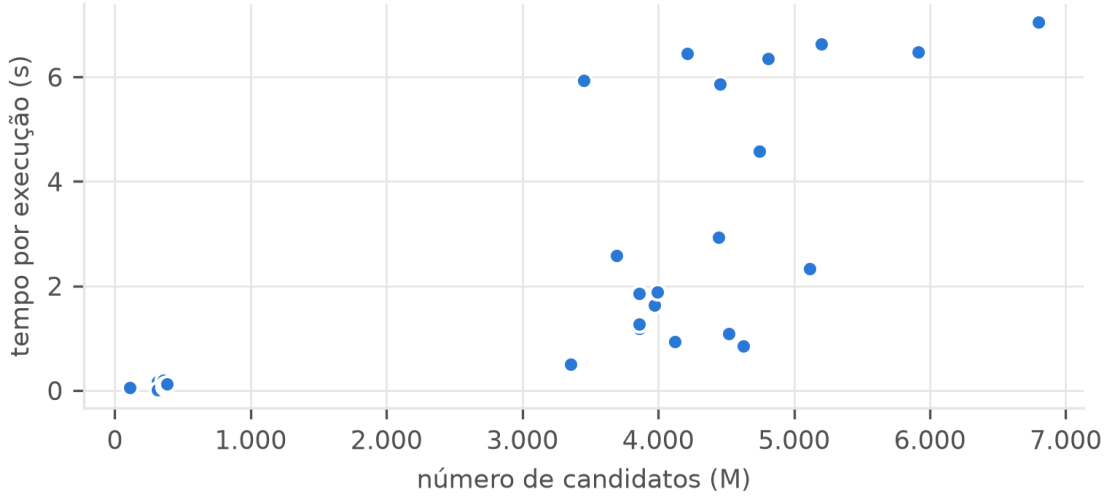


Figura 3. Tempo médio por execução (construtivo e VND) em função do número de candidatos.

8.4. Variabilidade

A Figura 4 mostra o desvio de cada execução em relação à média da instância. O construtivo varia pouco, com coeficiente de variação médio de 0,056% (máximo de 0,330%). Depois do VND, as 10 execuções terminam no mesmo valor em todas as instâncias menos

1 (I27, com CV de 0,022%). A aleatoriedade da fase B só muda a ordem de candidatos com arrependimentos parecidos, e o VND desfaz essas diferenças, porque os tipos, que são o que de fato define o ótimo local, vêm da fase A, que é determinística. O resultado final quase não depende da semente. Em compensação, a RCL da fase B não gera diversidade suficiente para sair do ótimo local, o que motiva algumas das melhorias da Seção 11.

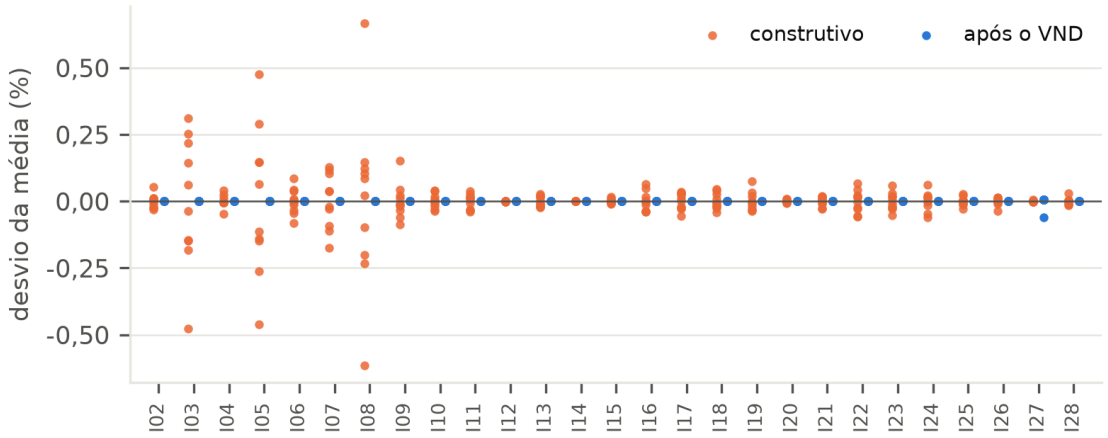


Figura 4. Desvio percentual de cada uma das 10 execuções em relação à média da instância, na solução inicial e após o VND.

8.5. Sensibilidade ao tamanho da RCL

Para escolher o tamanho da RCL, repetimos o experimento com $k \in \{1, 5, 25, 50\}$. Com $k = 1$ fizemos uma única execução, já que o construtivo é determinístico. A Tabela 5 mostra, em média sobre as instâncias, o gap da solução inicial e da final e o coeficiente de variação (CV) entre as 10 execuções.

Tabela 5. Sensibilidade ao tamanho da RCL: gaps médios e coeficiente de variação médio entre execuções.

k	Gap ini. (%)	Gap final (%)	Gap final melhor (%)	CV ini. (%)	CV final (%)	t (s)
1	19,97	0,735	0,735	0,000	0,000	2,62
5	19,96	0,735	0,735	0,048	0,000	2,58
10	19,96	0,734	0,732	0,056	0,001	2,56
25	19,96	0,733	0,732	0,109	0,001	2,58
50	20,00	0,734	0,732	0,129	0,001	2,58

Um k maior espalha mais as soluções do construtivo (o CV inicial vai de zero com $k = 1$ a cerca de 0,13% com $k = 50$), mas quase não muda a solução inicial média, porque o sorteio só troca a ordem de candidatos com arrependimentos parecidos. O valor final é praticamente o mesmo para todo k, com diferenças só na terceira casa decimal do gap, e o tempo também não muda: o VND converge para o mesmo ótimo local. Mantivemos $k = 10$ porque ele já gera variação mensurável entre as execuções sem piorar a solução inicial. Para melhorar o resultado final, a diversidade terá de vir de perturbações nos tipos das escolas (Seção 11), já que a RCL da fase B não a produz.

8.6. Informações da solução: locais utilizados e distribuição

Como não há custo de abertura e a folga é pequena, todas as escolas são usadas em todas as instâncias, menos em I20, a de maior folga, que deixa uma escola fechada (Tabela 3). O segundo tipo é usado quase sempre. Na melhor solução, em média 95,6% das escolas oferecem dois tipos (de 50% a 100% por instância), e em 21 instâncias todas oferecem dois. Isso acontece porque o modelo não cobra nada por um segundo tipo, e a heurística o abre sempre que isso reduz a distância. Se na prática um segundo tipo tiver custo (mais coordenadores, mais malotes de provas), basta acrescentar à função objetivo uma penalidade por escola com dois tipos.

A Figura 5 e a Tabela 6 mostram como os candidatos se distribuem na melhor solução da instância do grupo, I27 (5917 candidatos, 20 escolas e 197 salas), que fica 0,7% acima do melhor valor conhecido. As 20 escolas oferecem dois tipos, e cada tipo aparece em seis a oito escolas. Catorze escolas ficam cheias. A folga fica concentrada em E1 e E12 (52% e 63% de ocupação), cujas salas de M3e e M3d sobram porque os candidatos desses tipos têm escolas mais próximas.

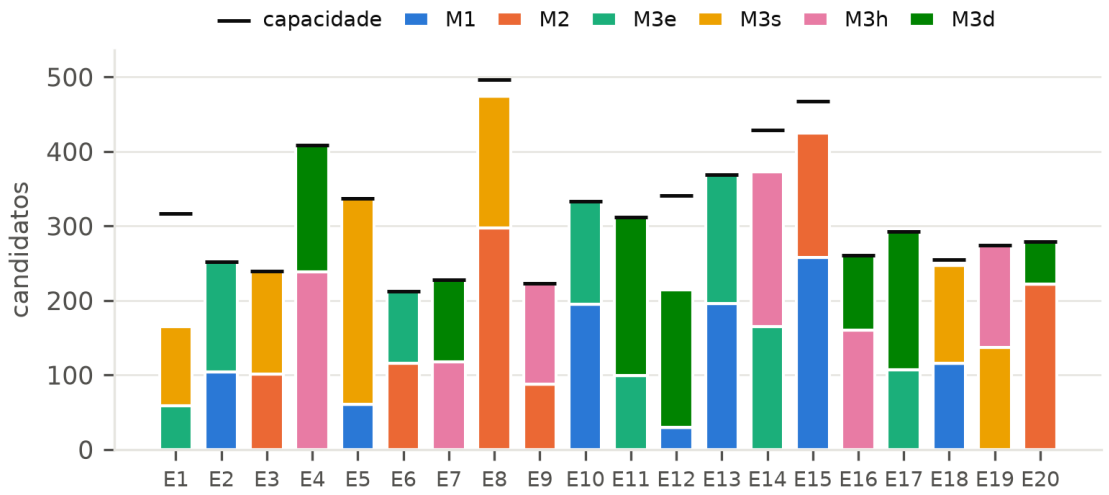


Figura 5. Candidatos por escola e tipo de prova na melhor solução de I27; o traço preto indica a capacidade da escola.

9. Exemplo passo a passo

Usamos a instância I03, com semente 0 e $k = 10$. Ela tem 316 candidatos e 12 escolas, cada uma com duas salas, de 15 e 16 lugares (31 lugares por escola). São 53 candidatos de cada tipo de M1 a M3s e 52 de M3h e de M3d. O registro completo da execução é gerado pela opção `--traco` do programa.

Na fase A, a alocação provisória estima para a escola E1 as demandas M1 = 5, M2 = 6, M3e = 8, M3s = 3, M3h = 3 e M3d = 4. E1 fica com M3e e M2, os dois de maior demanda estimada. Pelo *best-fit*, a sala de 15 lugares vai para M3e (falta 8) e a de 16, para M2 (falta 6). Repetindo o processo, todas as 12 escolas ficam com dois tipos. Mas só E7 e E10 escolhem M3d, que fica com 31 vagas para 52 candidatos, e M2 também fica descoberto. A etapa de cobertura então muda três salas: a sala 1 de E8 passa de M3s para M3d (faltavam 21 vagas de M3d), a sala 1 de E4 passa de M1 para M2 (faltavam 7 de M2)

Tabela 6. Resumo por escola da melhor solução de I27: capacidade, tipos oferecidos (com as vagas de cada um), candidatos, ocupação e distância média.

Escola	Cap.	Tipos (vagas)	Cand.	Ocup. (%)	Dist. média
E1	317	M3e (206) + M3s (111)	165	52,1	0,235
E2	252	M1 (104) + M3e (148)	252	100,0	0,258
E3	239	M2 (101) + M3s (138)	239	100,0	0,420
E4	408	M3h (238) + M3d (170)	408	100,0	0,158
E5	337	M1 (61) + M3s (276)	337	100,0	0,068
E6	212	M2 (116) + M3e (96)	212	100,0	0,009
E7	228	M3h (118) + M3d (110)	228	100,0	0,115
E8	496	M2 (319) + M3s (177)	474	95,6	0,187
E9	223	M2 (88) + M3h (135)	223	100,0	0,043
E10	333	M1 (195) + M3e (138)	333	100,0	0,359
E11	312	M3e (99) + M3d (213)	312	100,0	0,199
E12	341	M1 (82) + M3d (259)	214	62,8	0,137
E13	369	M1 (196) + M3e (173)	369	100,0	0,219
E14	429	M3e (172) + M3h (257)	373	86,9	0,082
E15	467	M1 (276) + M2 (191)	425	91,0	0,028
E16	261	M3h (160) + M3d (101)	261	100,0	0,513
E17	292	M3e (107) + M3d (185)	292	100,0	0,010
E18	255	M1 (124) + M3s (131)	247	96,9	0,033
E19	274	M3s (137) + M3h (137)	274	100,0	0,011
E20	279	M2 (222) + M3d (57)	279	100,0	0,285

e a sala 2 de E12 passa de M1 para M3d (faltavam 5). Cada mudança só é aceita porque o tipo de origem continua coberto. No fim, cada tipo tem de 61 a 63 vagas para 52 ou 53 candidatos.

Na fase B, os 10 candidatos de maior arrependimento formam a RCL (C213, C294, C172, ..., com arrependimentos de 4,03 a 3,65). O primeiro sorteado é C287, do tipo M3s. As quatro escolas mais próximas dele (E6, E9, E1 e E5) não têm sala de M3s, e ele vai para a sala 1 de E3, a 8,12 de distância. C287 sai da RCL e entra C290, o 11º da fila. O processo continua até os 316 candidatos estarem alocados, com custo inicial de 1064,53.

No VND, a N1 não encontra melhora, já que a fase B pôs cada candidato na escola mais próxima com vaga. A N2 faz 40 trocas e leva o custo a 991,37; numa delas, C255 (E3) troca com C206 (E4) e o custo cai 6,32. A N3 encontra três cadeias, como C288 de E3 para E7, C191 de E7 para E2 e C35 de E2 para E3 (ganho de 0,30), e o custo vai a 990,93. A N4 não melhora, e a N5 troca os tipos das salas de E4 (16 lugares de M2 e 15 de M3s), com ganho de 2,41, o que faz a VND voltar para a N1. Na execução inteira, a VND aplica 16 movimentos da N1, 63 da N2, 19 da N3, 10 da N5 e 11 da N7. O maior ganho vem de uma troca da N7 que leva M3h de E5 para E2 e M3e de E2 para E5: 36 candidatos mudam de escola, e o custo cai 37,93. A busca termina em 865,52, uma melhoria de 18,7% sobre a solução inicial, 3,0% acima do ótimo provado pelo MIP (839,97).

10. Onde a abordagem não vai bem

Os piores resultados são I07 (8,3% acima do ótimo provado pelo MIP), I08 (4,8%) e I03 (3,0%). As três são pequenas e têm ótimo conhecido. Nas instâncias grandes a referência é quase sempre a própria heurística, e o que podemos afirmar sobre elas é o gap até o LB (Seção 8). Encontramos três causas para esses casos.

A primeira é que o ótimo distribui os tipos de um jeito muito diferente. I07 tem 370 candidatos e 12 escolas, cada uma com duas a quatro salas de 4 a 16 lugares. Com-

parando os tipos da melhor solução da heurística com os do ótimo do MIP, só uma das 12 escolas oferece o mesmo par de tipos nas duas. Para chegar ao ótimo seria preciso mudar os tipos de várias escolas ao mesmo tempo, mas cada movimento de N4 a N7 mexe em no máximo duas escolas e só é aceito se melhorar a solução por conta própria. Com poucas salas por escola, e de tamanhos variados, a divisão das vagas entre os dois tipos é grosseira, e muitos dos movimentos intermediários pioram a solução.

A segunda é que a avaliação das vizinhanças de tipos é gulosa. As chegadas só ocupam as vagas novas, e os despejos vão para a vaga mais próxima disponível, sem reorganizar o resto da alocação. Um movimento de tipo que só compensaria depois de reotimizar a alocação inteira acaba rejeitado.

A terceira está na fase B: o arrependimento é calculado entre as duas escolas mais próximas, mesmo que elas não ofereçam o tipo do candidato. No exemplo, C287 está entre os 10 primeiros da fila, mas nenhuma das quatro escolas mais próximas dele tem sala de M3s, e o arrependimento não reflete o que ele realmente perde.

Há ainda uma instância sem solução. I01 tem 100 candidatos, 5 escolas e 20 salas de 5 ou 6 lugares, num total de 105 vagas. A demanda é de 16 ou 17 candidatos por tipo, e um tipo com 17 candidatos precisa de três salas, sendo pelo menos duas de 6 lugares ($6 + 6 + 5$). Só existem quatro salas de 6, o que atende no máximo dois desses tipos, então a instância não tem solução nem sem limite de tipos por escola. A folga total de 5% esconde o problema, que só aparece quando se olha o tamanho de cada sala. A fase A detecta a inviabilidade pelo reparo exato, sem gastar tempo de busca.

11. Conclusões

Propusemos uma heurística construtiva em duas fases e uma VND com sete vizinhanças para o problema de localização de locais de prova em que cada sala aplica um único tipo de prova e cada escola oferece no máximo dois. A etapa de cobertura da fase A, com o reparo exato para quando a cobertura gulosa trava, faz a construção funcionar em todas as instâncias viáveis, e todas as soluções geradas são viáveis, sem precisar de penalização. Nas 27 instâncias viáveis da turma, o método melhora a solução inicial em 15,29% e termina, em média, 0,73% acima do melhor valor conhecido e 9,40% acima do limite inferior, em 2,56 s por execução. Em 17 das 27 instâncias a heurística encontra soluções melhores que o MIP de 60 s.

A primeira dificuldade foi a variedade das instâncias. Versões anteriores usavam seis formatos de arquivo, e a versão final ainda tem cabeçalhos diferentes, arquivos sem dados, duplicatas e distâncias em unidades diferentes; precisamos de um leitor tolerante a essas variações e de comparar os resultados por gap. Cobrir os tipos com salas indivisíveis também deu trabalho: a cobertura gulosa trava em 3 das 27 instâncias, o que nos levou ao reparo exato. Com a cobertura justa, mudar um tipo inteiro de uma escola raramente é viável, e foi por isso que criamos a N7, que troca os tipos de duas escolas, e a N5, que troca os tipos de duas salas. Por fim, faltam ótimos provados nas instâncias grandes, onde o MIP de 60 s fica pior que a heurística e o limite inferior às vezes é fraco.

Algumas melhorias possíveis:

- avaliar os vizinhos de N4 a N7 reotimizando a alocação, pelo menos para os mais promissores segundo a avaliação gulosa;

- acrescentar uma vizinhança que rotacione tipos entre três escolas, ou uma busca local iterada (ILS) que perturbe os tipos de algumas escolas e reaplique a VND, para escapar dos ótimos locais de tipos (a primeira causa da Seção 10) e criar diversidade de fato entre as execuções;
- calcular o arrependimento só entre as escolas que oferecem o tipo do candidato;
- penalizar o segundo tipo na função objetivo, para que ele só seja usado onde o ganho em distância compensa.

Referências

Ahuja, R. K., Magnanti, T. L., and Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs.

Feo, T. A. and Resende, M. G. C. (1995). Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6(2):109–133.

Glover, F. (1996). Ejection chains, reference structures and alternating path methods for traveling salesman problems. *Discrete Applied Mathematics*, 65(1–3):223–253.

Huangfu, Q. and Hall, J. A. J. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142.

Mladenović, N. and Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research*, 24(11):1097–1100.

Oliveira, T. E. d., Martinelli Pinto, R., and Milanez, B. (2021). Solving a university admission exam location problem: An application in Brazil. Technical report, Pontifícia Universidade Católica do Rio de Janeiro, Rio de Janeiro. Disponível em `https://www.maxwell.vrac.puc-rio.br/52465/52465.PDF`.

Virtanen, P. et al. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272.